

SQL avancé avec PostgreSQL

Questionnaire de fin de module couvrant les notions N1 à N41 des 5 séances.

40 questions

Durée conseillée : 60 min

Sans document ni ordinateur

Noté sur 20

Nom :

Prénom :

Groupe / promotion :

Date :

Consignes

- Répondez directement sur le sujet en cochant ou en noircissant la ou les cases choisies.
- Pour corriger une réponse, barrez-la nettement d'une croix et entourez la bonne case.
- Toutes les requêtes sont écrites pour PostgreSQL. Les tables citées sont celles des démonstrations du module (joueurs, équipes, jeux, aventuriers...).

QCM

Une seule bonne réponse. Cases rondes ☐.

Vrai / Faux

Cochez Vrai ou Faux. Cases rondes ☐.

Cases à cocher

Une ou plusieurs bonnes réponses. Cases carrées ☐.

Barème : 1 point par question, soit 40 points ramenés sur 20. Pour les questions à cases à cocher, le point n'est accordé que si **toutes** les bonnes cases sont cochées **et aucune** mauvaise. Une réponse fausse ne retire pas de point.

Séance	Thème	Questions	Points
S1	Fondamentaux et requêtage avancé (N1 → N9)	Q1 → Q8	/ 8
S2	Agrégations, JSONB, vues et fonctions (N10 → N17)	Q9 → Q16	/ 8
S3	Optimisation et index (N18 → N25)	Q17 → Q24	/ 8
S4	Procédures stockées et triggers (N26 → N33)	Q25 → Q32	/ 8
S5	Transactions et sécurité (N34 → N41)	Q33 → Q40	/ 8

Total : _____ / 40

Note : _____ / 20

S1

Fondamentaux et requêtage avancé

Notions N1 à N9 · psql, types, jointures, agrégats, CTE, fenêtrage

Q1

QCM

N1

Dans psql, l'invite passe de `cours_sql=#` à `cours_sql-#` après avoir tapé une requête et appuyé sur Entrée. Que se passe-t-il ?

- ☐ A La connexion au serveur a été perdue
- ☐ B psql attend la suite de l'instruction : le point-virgule final manque
- ☐ C La session n'a plus les droits de superutilisateur
- ☐ D La transaction en cours a échoué

Q2

Cases à cocher

N2

On crée la colonne `prix` d'un catalogue et sa clé primaire. Quelles affirmations sont exactes ?

- ☐ A `NUMERIC(10,2)` est le type adapté pour stocker un prix
- ☐ B `REAL` garantit que $0.1 + 0.2$ vaut exactement 0.3
- ☐ C `CHECK (prix >= 0)` fait refuser par la base toute ligne au prix négatif
- ☐ D `GENERATED ALWAYS AS IDENTITY` est la syntaxe standard à privilégier à `SERIAL`

Q3

Vrai / Faux

N3

La requête suivante ne renvoie que des joueurs ayant marqué plus de 3 buts.

```
SELECT nom FROM joueurs
WHERE poste = 'attaquant' AND buts > 3 OR poste = 'milieu';
```

- ☐ Vrai
- ☐ Faux

Q4

QCM

N4

On veut lister **toutes** les équipes avec leur nombre de joueurs, y compris celles qui n'en ont aucun. Quelle jointure compléter ?

```
SELECT e.nom, COUNT(j.id)
FROM equipes e _____ joueurs j ON j.equipe_id = e.id
GROUP BY e.nom;
```

- ☐ A `INNER JOIN`
- ☐ B `LEFT JOIN`
- ☐ C `CROSS JOIN`
- ☐ D `NATURAL JOIN`

Q5

Cases à cocher

N5

Parmi ces affirmations sur `GROUP BY` et `HAVING`, lesquelles sont exactes ?

- ☐ A `WHERE` filtre les lignes avant le regroupement
- ☐ B `HAVING` peut filtrer sur un agrégat, par exemple `HAVING COUNT(*) > 3`
- ☐ C `HAVING` est évalué avant `WHERE`
- ☐ D `WHERE COUNT(*) > 3` est une écriture valide

Q6

Cases à cocher

N6 · N7

Parmi ces affirmations sur les CTE, lesquelles sont exactes ?

- ☐ A Une CTE n'existe que le temps de la requête qui la suit
- ☐ B Chaque CTE doit être précédée de son propre mot-clé `WITH`
- ☐ C Une CTE récursive s'écrit `WITH RECURSIVE` et combine une partie d'ancrage et une partie récursive par `UNION ALL`
- ☐ D Sur des données qui forment une boucle ($A \rightarrow B \rightarrow A$), une CTE récursive s'arrête toujours d'elle-même

Q7

QCM

N8

Les scores triés par ordre décroissant sont 100, 90, 90, 80. Quels rangs `RANK() OVER (ORDER BY score DESC)` attribue-t-il ?

- ☐ A 1, 2, 3, 4
- ☐ B 1, 2, 2, 3
- ☐ C 1, 2, 2, 4
- ☐ D 1, 1, 2, 3

Q8

Cases à cocher

N8 · N9

Parmi ces affirmations sur les fonctions de fenêtrage, lesquelles sont exactes ?

- ☐ A `LAG(ca) OVER (ORDER BY mois)` renvoie le chiffre d'affaires du mois précédent
- ☐ B `SUM(ca) OVER (ORDER BY mois)` calcule un cumul progressif
- ☐ C On peut écrire `WHERE rang <= 3` si `rang` est l'alias d'un `RANK() OVER (...)` du même `SELECT`
- ☐ D Avec un `ORDER BY` et sans cadre précisé, les lignes à égalité sont cumulées ensemble (cadre `RANGE`)

Q9

QCM

N10

On veut, dans une même requête, deux colonnes : le nombre de ventes sur PC et le nombre de ventes sur PS5. Quelle écriture convient ?

- ☐ A `COUNT(*) AS pc, COUNT(*) AS ps5` avec `WHERE plateforme = 'PC' AND plateforme = 'PS5'`
- ☐ B `COUNT(plateforme = 'PC') AS pc, COUNT(plateforme = 'PS5') AS ps5`
- ☐ C `COUNT(*) FILTER (WHERE plateforme = 'PC') AS pc, COUNT(*) FILTER (WHERE plateforme = 'PS5') AS ps5`
- ☐ D `COUNT(*)` avec `GROUP BY plateforme HAVING plateforme IN ('PC', 'PS5')`, sur une seule ligne

Q10

Vrai / Faux

N11

Avec `GROUP BY ROLLUP (...)`, tester `colonne IS NULL` suffit toujours pour reconnaître une ligne de sous-total.

- ☐ Vrai ☐ Faux

Q11

QCM

N12

Pour chaque jeu, on veut ses 3 meilleurs mois de ventes avec une sous-requête qui fait référence à la table de gauche (`WHERE v.jeu_id = j.id ... LIMIT 3`). Que faut-il écrire ?

- ☐ A `FROM jeux j CROSS JOIN LATERAL (sous-requête) m`
- ☐ B `FROM jeux j NATURAL JOIN (sous-requête) m`
- ☐ C `FROM jeux j, (sous-requête) m`, sans autre mot-clé
- ☐ D `FROM jeux j UNION (sous-requête)`

Q12

Cases à cocher

N13

La colonne `details` est de type `JSONB`. Quelles affirmations sont exactes ?

- ☐ A `details ->> 'pegi'` renvoie du texte
- ☐ B `details -> 'pegi'` renvoie du texte
- ☐ C `details ->> 'pegi' > '12'` compare les valeurs numériquement
- ☐ D `details @> '{"multijoueur": true}'` teste si le document contient cette paire

Q13

Vrai / Faux

N14

Une vue ne stocke pas de données : sa requête est réexécutée à chaque lecture.

- ☐ Vrai ☐ Faux

Q14

Cases à cocher

N15

Parmi ces affirmations sur les vues matérialisées, lesquelles sont exactes ?

- ☐ A Leurs données restent figées jusqu'au prochain `REFRESH MATERIALIZED VIEW`
- ☐ B `REFRESH MATERIALIZED VIEW CONCURRENTLY` exige un index unique sur la vue
- ☐ C Elles se mettent à jour automatiquement à chaque `INSERT` dans les tables sources
- ☐ D On peut créer des index sur une vue matérialisée

Q15

Vrai / Faux

N16

`CREATE OR REPLACE FUNCTION` permet de changer le type de retour d'une fonction existante.

- ☐ Vrai ☐ Faux

Q16

QCM

N17

Une fonction PL/pgSQL lit la table `joueurs` sans la modifier. Quelle catégorie de volatilité faut-il déclarer ?

- ☐ A `IMMUTABLE`
- ☐ B `STABLE`
- ☐ C `VOLATILE`
- ☐ D `STRICT`

Q17

Vrai / Faux

N18

EXPLAIN ANALYZE DELETE FROM ecoutes WHERE ...; affiche le plan sans supprimer aucune ligne.

- ☐ Vrai ☐ Faux

Q18

QCM

N19

Une requête ramène environ 60 % des lignes d'une grande table, alors qu'un index B-tree existe sur la colonne filtrée. Quel mode de lecture le planificateur choisira-t-il très probablement ?

- ☐ A Seq Scan
- ☐ B Index Scan
- ☐ C Index Only Scan
- ☐ D Aucun : la requête échoue

Q19

QCM

N20

Juste après un import massif, un plan affiche rows=12 estimé pour rows=480000 réel. Quelle est l'interprétation la plus pertinente ?

- ☐ A Les statistiques sont obsolètes : il faut lancer ANALYZE sur la table
- ☐ B La table est corrompue : il faut la recréer
- ☐ C Il manque un index : il faut en créer un sur chaque colonne
- ☐ D L'écart est normal et n'a aucun effet sur le choix du plan

Q20

Cases à cocher

N21

Un index composite existe sur ecoutes (utilisateur_id, ecoute_le) . Quelles conditions peuvent l'exploiter efficacement ?

- ☐ A WHERE utilisateur_id = 42
- ☐ B WHERE utilisateur_id = 42 AND ecoute_le >= '2025-01-01'
- ☐ C WHERE ecoute_le >= '2025-01-01' , seule
- ☐ D WHERE ecoute_le = '2025-03-01' OR utilisateur_id = 42

Q21

QCM

N22

On a créé l'index `CREATE INDEX ON utilisateurs (LOWER(pseudo));`. Quelle requête peut l'utiliser ?

- ☐ A `WHERE UPPER(pseudo) = 'USER_4242'`
- ☐ B `WHERE pseudo ILIKE 'user_4242'`
- ☐ C `WHERE LOWER(pseudo) = 'user_4242'`
- ☐ D `WHERE pseudo = 'user_4242'`

Q22

Vrai / Faux

N23

Un index GIN sur une colonne `JSONB` accélère `details @> '{"pegi": 18}'`, mais pas `details ->> 'pegi' = '18'`.

- ☐ Vrai
- ☐ Faux

Q23

Cases à cocher

N24

La colonne `cree_le` (de type `timestamp`) a un index B-tree simple. Quelles conditions empêchent son utilisation ?

- ☐ A `WHERE EXTRACT(YEAR FROM cree_le) = 2025`
- ☐ B `WHERE cree_le >= '2025-01-01' AND cree_le < '2026-01-01'`
- ☐ C `WHERE cree_le::date = '2025-03-01'`
- ☐ D `WHERE cree_le + INTERVAL '1 day' > now()`

Q24

Vrai / Faux

N25

Un index qui n'apporte aucun gain mesuré doit être supprimé, car chaque index ralentit les écritures et consomme du stockage et de l'énergie.

- ☐ Vrai
- ☐ Faux

Q25

Cases à cocher

N26

Parmi ces affirmations sur les procédures (`CREATE PROCEDURE`), lesquelles sont exactes ?

- ☐ A Une procédure s'appelle avec `CALL acheter(1, 42);`
- ☐ B Une procédure s'appelle avec `SELECT acheter(1, 42);`
- ☐ C Une procédure peut transmettre une valeur à l'appelant via un paramètre `INOUT`
- ☐ D Une procédure renvoie son résultat avec `RETURN valeur;`

Q26

QCM

N27

Quel est le principal danger de ce bloc dans une procédure ?

```
EXCEPTION  
WHEN OTHERS THEN  
    RAISE NOTICE 'Une erreur est survenue';
```

- ☐ A Il avale toutes les erreurs : l'appelant croit que tout s'est bien passé
- ☐ B Il provoque une erreur de syntaxe
- ☐ C Il annule systématiquement toute la base
- ☐ D Il empêche la procédure de s'exécuter

Q27

Vrai / Faux

N28

Une procédure qui contient un `COMMIT` peut être appelée à l'intérieur d'un bloc `BEGIN; ... COMMIT;` ouvert par l'appelant.

- ☐ Vrai ☐ Faux

Q28

Cases à cocher

N29

Quels éléments font partie de la mise en place d'un trigger dans PostgreSQL ?

- ☐ A Une fonction déclarée `RETURNS trigger`
- ☐ B Une instruction `CREATE TRIGGER ... EXECUTE FUNCTION ...`
- ☐ C Un moment (`BEFORE` / `AFTER`) et un événement (`INSERT` , `UPDATE` , `DELETE`)
- ☐ D Un appel explicite par `CALL` après chaque écriture

Q29

QCM

N30

Dans un trigger `BEFORE INSERT ... FOR EACH ROW`, la fonction se termine par `RETURN NULL;`. Que se passe-t-il ?

- ☐ A La ligne est insérée avec toutes ses colonnes à NULL
- ☐ B L'insertion de la ligne est annulée silencieusement, sans erreur
- ☐ C Une erreur est levée et la transaction échoue
- ☐ D Le trigger est ignoré et la ligne est insérée normalement

Q30

QCM

N31

On veut tenir un journal d'audit qui n'enregistre que les modifications réellement appliquées. Quel type de trigger choisir ?

- ☐ A `BEFORE ... FOR EACH ROW`
- ☐ B `AFTER ... FOR EACH ROW`
- ☐ C `INSTEAD OF` sur la table
- ☐ D Aucun : c'est impossible avec un trigger

Q31

Cases à cocher

N32

Parmi ces affirmations sur les triggers d'instruction, les tables de transition et `WHEN`, lesquelles sont exactes ?

- ☐ A Les tables de transition (`REFERENCING NEW TABLE AS ...`) ne sont disponibles qu'avec `AFTER`
- ☐ B Un trigger `FOR EACH STATEMENT` ne se déclenche que si au moins une ligne est modifiée
- ☐ C `WHEN (OLD IS DISTINCT FROM NEW)` évite d'exécuter un trigger `UPDATE` quand rien ne change
- ☐ D Les tables de transition sont compatibles avec une liste de colonnes (`UPDATE OF prix`)

Q32

Vrai / Faux

N33

Un trigger se justifie quand une règle doit être garantie quel que soit le programme qui écrit dans la table (audit, intégrité, normalisation).

- ☐ Vrai
- ☐ Faux

Q33

QCM

N34

Dans une transaction ouverte par `BEGIN`, une instruction échoue. On tape ensuite `COMMIT;`. Que se passe-t-il ?

- ☐ A Les instructions réussies avant l'erreur sont validées
- ☐ B Le `COMMIT` se transforme en `ROLLBACK` : rien n'est validé
- ☐ C Le `COMMIT` valide tout, y compris l'instruction en erreur
- ☐ D PostgreSQL redémarre la transaction automatiquement

Q34

QCM

N35

Deux transactions lisent les mêmes données puis écrivent chacune une ligne différente, et laissent ensemble la base dans un état incohérent (*write skew*). Quel niveau d'isolation l'empêche à lui seul ?

- ☐ A `READ UNCOMMITTED`
- ☐ B `READ COMMITTED`
- ☐ C `REPEATABLE READ`
- ☐ D `SERIALIZABLE`

Q35

Vrai / Faux

N36

Les verrous posés par un `UPDATE` sont conservés jusqu'au `COMMIT` ou au `ROLLBACK` de la transaction.

- ☐ Vrai
- ☐ Faux

Q36

Cases à cocher

N37

Quelles affirmations sur les interblocages (*deadlocks*) sont exactes ?

- ☐ A Verrouiller toujours les lignes dans le même ordre (par identifiant croissant, par exemple) réduit le risque
- ☐ B L'application doit prévoir de relancer une transaction annulée (erreur 40P01)
- ☐ C Augmenter le nombre maximal de connexions les fait disparaître
- ☐ D PostgreSQL détecte l'interblocage et annule l'une des transactions

Q37

Vrai / Faux

N38

Pour éviter les erreurs de droits, l'application doit se connecter avec le compte `postgres`.

- ☐ Vrai
- ☐ Faux

Q38

Cases à cocher

N39

La Row Level Security est activée sur une table, sans `FORCE ROW LEVEL SECURITY`. Quels rôles ignorent ses politiques ?

- ☐ A Un superutilisateur comme `postgres`
- ☐ B Un rôle doté de l'attribut `BYPASSRLS`
- ☐ C Le propriétaire de la table
- ☐ D Le rôle applicatif, qui a seulement le droit `SELECT` sur la table

Q39

QCM

N40

Quelle précaution est indispensable dans une fonction `SECURITY DEFINER` ?

- ☐ A `SET search_path = pg_temp, public`
- ☐ B `SET search_path = public, pg_temp`
- ☐ C `GRANT ALL ON ALL TABLES IN SCHEMA public TO PUBLIC`
- ☐ D La déclarer `IMMUTABLE`

Q40

Cases à cocher

N41

Quelles mesures protègent réellement contre l'injection SQL ?

- ☐ A Utiliser des requêtes paramétrées
- ☐ B Échapper les apostrophes « à la main »
- ☐ C Filtrer les mots-clés dangereux (`DROP`, `DELETE` ...)
- ☐ D Pour un nom de colonne choisi par l'utilisateur, le valider contre une liste blanche puis l'insérer avec `format('%I', nom)`
- ☐ E Appliquer le moindre privilège au rôle applicatif, pour limiter l'impact d'une faille